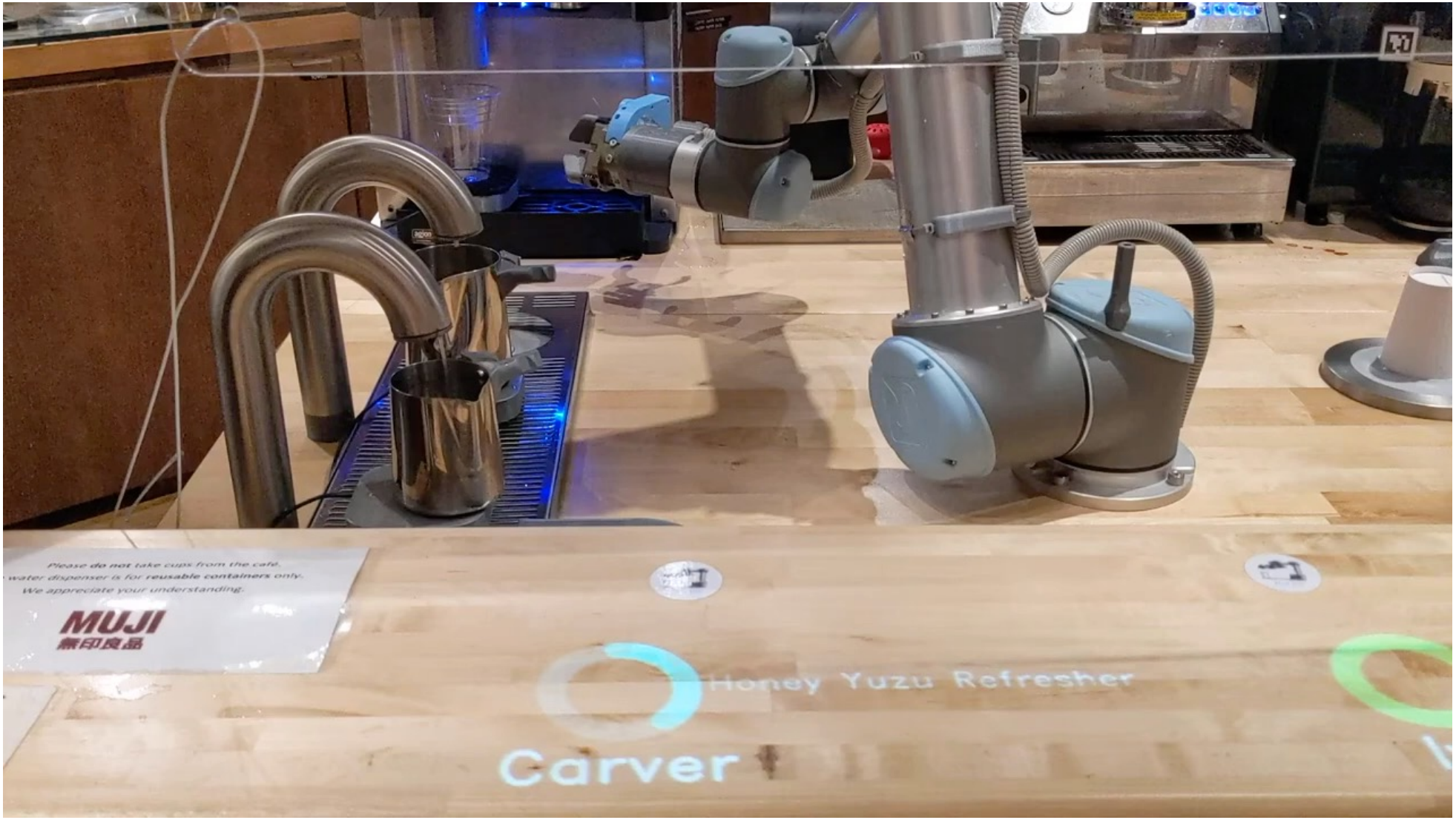
B3. Planning under Uncertainty

Hanna Kurniawati

Australian National University | School of Computing
ROBOTICS @ ANU COMPUTING
SMARTSAT Cooperative Research Centre

By now, we know…

- Hierarchy in Robot Planning
  - Symbolic Planning
  - Task – Motion Planning
  - Motion Planning
- Assumption: Perfect model
- This lecture: Relax the perfect model assumption
  - Aim: Robus & resilient robot operation



https://www.youtube.com/watch?v=DHqLkGPrzso

Planning under Uncertainty

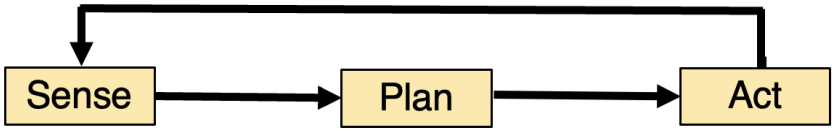
Goal: Generate strategies for the robot to achieve its objective (e.g., reach a goal, serve Yuzu Honey Refresher, gather good data, etc.) **well**, despite various types of uncertainty

Well: Can be many criteria

Planning under Uncertainty

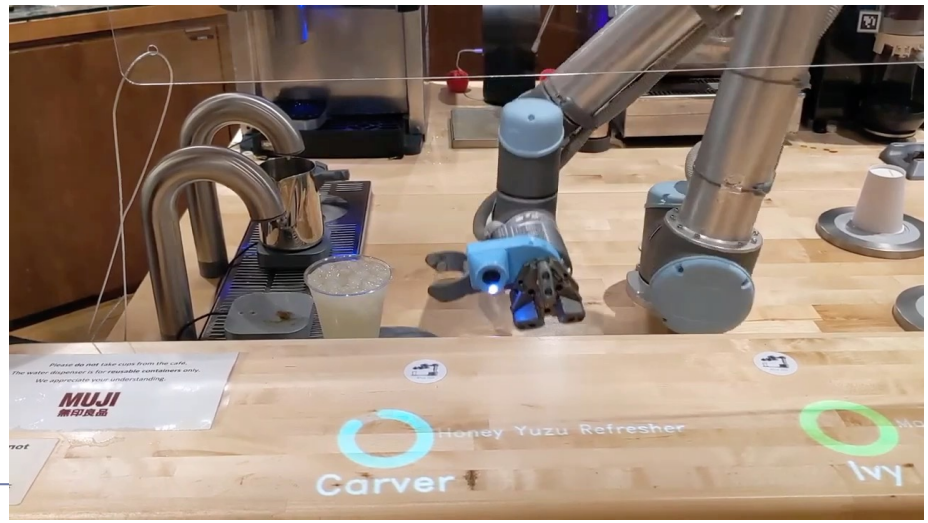
Sequential decision-making ;
Have a model to predict outcomes of actions/sensing before execution

Classify: The act and the sense



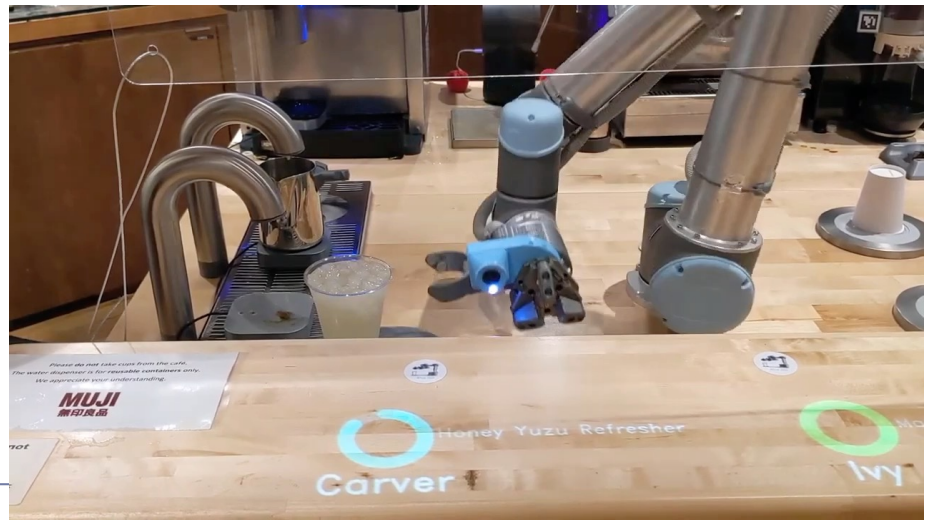
A bit of terminology

- Uncertainty in the act
  - Deterministic vs non-deterministic effects of actions
  - Does an action always move the robot from a state to exactly one (other/same) state?
    - Yes: Deterministic
    - No: Non-deterministic
  - The video example?



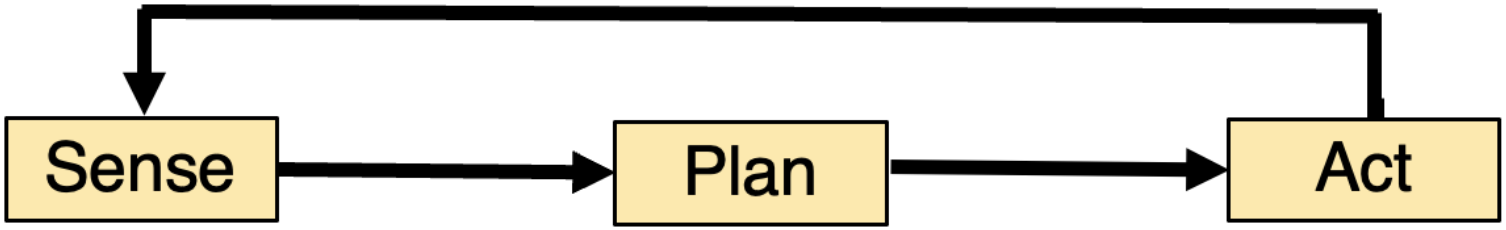
A bit of terminology

- Uncertainty in the sense
  - Fully observed vs partially-observed
  - Does the robot know the state of the world exactly?
    - Yes: Fully observed
    - No: Partially observed
  - The video example?



Seems only 2 types of uncertainty, but…

- Covers all interactions of the planner with the system outside of the planner, incl. the environment



Seems only 2 types of uncertainty, but…

- Can represent various types of uncertainty, e.g.:
  - Actuators errors
  - Measurement errors
  - Errors in processing sensor data
  - Errors in modelling the agents' own dynamics and the environment
  - Uncertainty in human behaviour when a robot collaborates with humans
  - Initially unknown physical parameters (e.g., coef. friction, weight, etc.)
- Trick is in problem modelling

This session

- Frameworks for planning under uncertainty
  - Markov Decision Processes (MDPs)
  - Partially Observable MDPs (POMDPs)
  - Reinforcement Learning
- Offline Solving: Value Iteration
  - Value Iteration for MDP
- Online Solving: MCTS-based
  - MDP

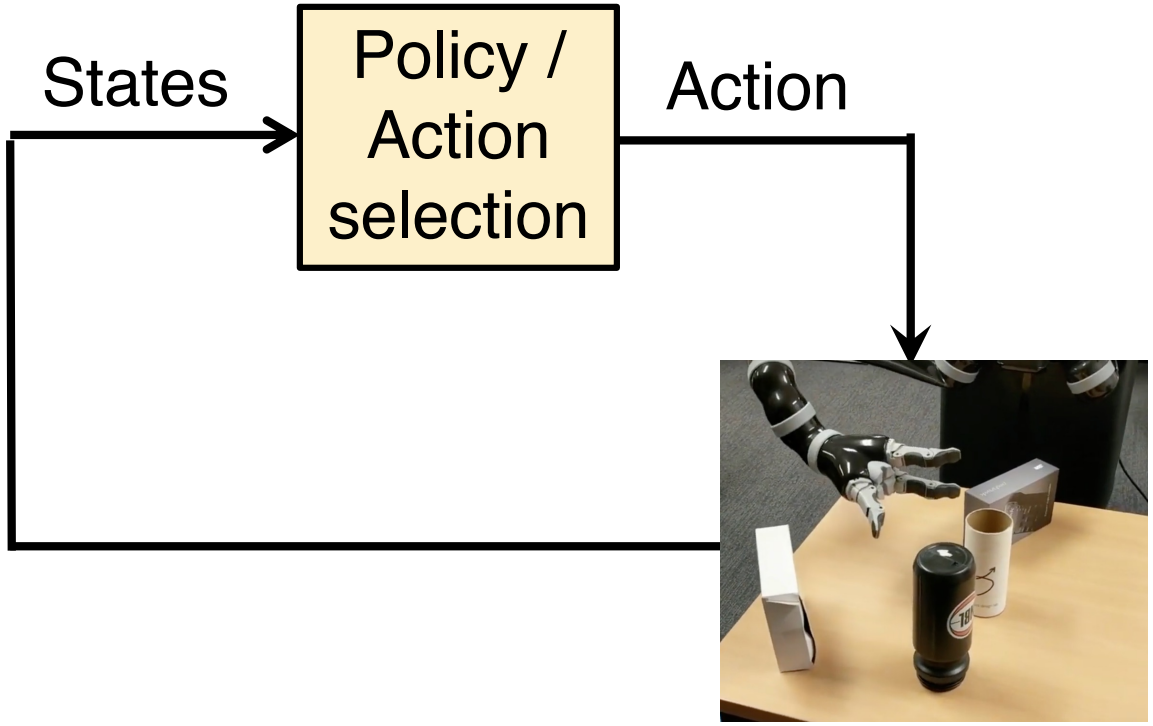
Frameworks for planning under uncertainty

- Markov Decision Processes (MDPs): When the effects of actions are non-deterministic
- Partially Observable MDPs (POMDPs): Uncertainty the effects of actions are non-deterministic and the world is partially observed
- Reinforcement Learning: When the models are not initially available

Markov Decision Processes (MDPs)

- Decisions are made at discrete time
- At any given time step, the agent's state is known exactly (fully observed)
- But, the effects of actions are not known exactly before the action is executed (non-deterministic action effects)
  - How does this affect the format of the planning solution? Would a path / sequence of actions be a sufficient plan?
  - No, we need to compute a mapping from states to actions, namely a policy

Markov Decision Processes (MDPs)



T(s, a, s'): Transition function, cond. prob. func P(s' | s, a)
where s, s': States, a: action

R: Reward function

Defining an MDP Problem / Model

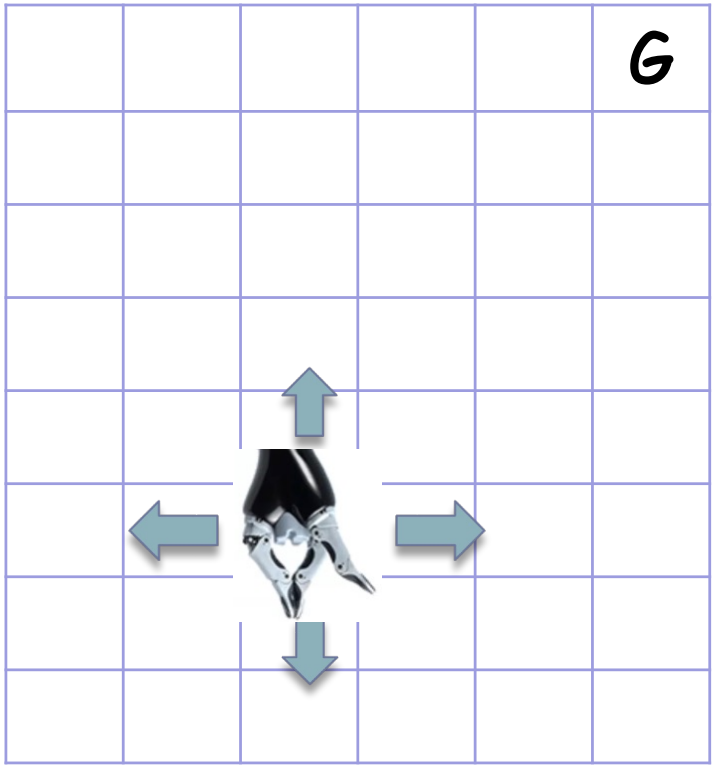
- Formally defined as a 4-tuples

  (S, A, T, R):

  - S: State space.
  - A: Action space.
  - T: transition function.

    $T(s, a, s') = P(S_{t+1} = s' \mid S_t = s, A_t = a)$.
  - R: Reward function.

    R(s) or R(s, a) or R(s, a, s')



The states & actions can be continuous spaces
In this case, the transition is the pdf

Markov Decision Processes (MDPs)

- Solving an MDP = finding the best policy π^*
- What does "best" means?
- Many objectives have been proposed
- Some commonly used are the policy π that maximizes
  - Myopic: $E[R_t(s,a)|\pi, s]$
  - Finite horizon: $E\left[\sum_{t=0}^{k} R_t(s,a) \,\middle|\, \pi, s_0\right]$
  - Infinite horizon: $E\left[\sum_{t=0}^{inf} \gamma^t R_t(s,a) \,\middle|\, \pi, s_0\right]$

s_0: initial state
s: state
a: action
$R_t(s,a)$: immediate reward
γ: discount factor with $0 < \gamma < 1$

Difference in Optimal Policy?

- Finite vs Infinite Horizon?

$$E\left[\sum_{t=0}^{k} R_t(s,a) \,\middle|\, \pi, s_0\right] \quad \text{VS} \quad E\left[\sum_{t=0}^{inf} \gamma^t R_t(s,a) \,\middle|\, \pi, s_0\right]$$

s_0: initial state
s: state
a: action
$R_t(s,a)$: immediate reward
γ: discount factor with $0 < \gamma < 1$

The optimal policy is stationary

Value of a policy

- The expected total reward the agent will gather if it executes a policy π is called the value function and is denoted by V_π.

$$V_\pi(s) = R(s,a) + \gamma \sum_{s'} T(s,\pi(s),s')V_\pi(s')$$

Immediate reward

Expected total future reward

Optimal Value function

- Each policy has a corresponding value function.
- An optimal policy π^* is a policy whose corresponding value function is the maximum.
  - Meaning at each state, $V_{\pi^*}(s)$ is higher than (or at least the same as) V_π for any other policy π.
  - V_{π^*} is often simplified as V^*
  - In other words,

Q(s, a)

$$V^*(s) = \max_a \left(R(s,a) + \gamma \sum_{s'} T(s,a,s') V^*(s') \right)$$

A note

- There is a unique optimal value function V*
- But, there might be more than one optimal policy
  - If we know V*, the optimal policy can be generated easily

$$\pi^*(s) = \underset{a}{\operatorname{argmax}} \left(R(s,a) + \gamma \sum_{s'} T(s,a,s') V^*(s') \right)$$

Markov Decision Processes (MDPs)

- Decisions are made at discrete time
- At any given time step, the agent's state is known exactly (fully observed)
- But, the effects of actions are not known exactly before the action is executed (non-deterministic action effects)
  - The solution format is a policy –that is, a mapping from states to actions

This session

- Frameworks for planning under uncertainty
  - ✓Markov Decision Processes (MDPs)
  - Partially Observable MDPs (POMDPs)
  - Reinforcement Learning
- Offline Solving: Value Iteration
  - Value Iteration for MDP
- Online Solving: MCTS-based
  - MDP

Partially Observable Markov Decision Processes (POMDPs)

- Discrete time step
- The effects of actions are not known exactly before the action is executed (non-deterministic action effects)
- In addition, at any given time, the agent's state is NOT known exactly (partially observed)

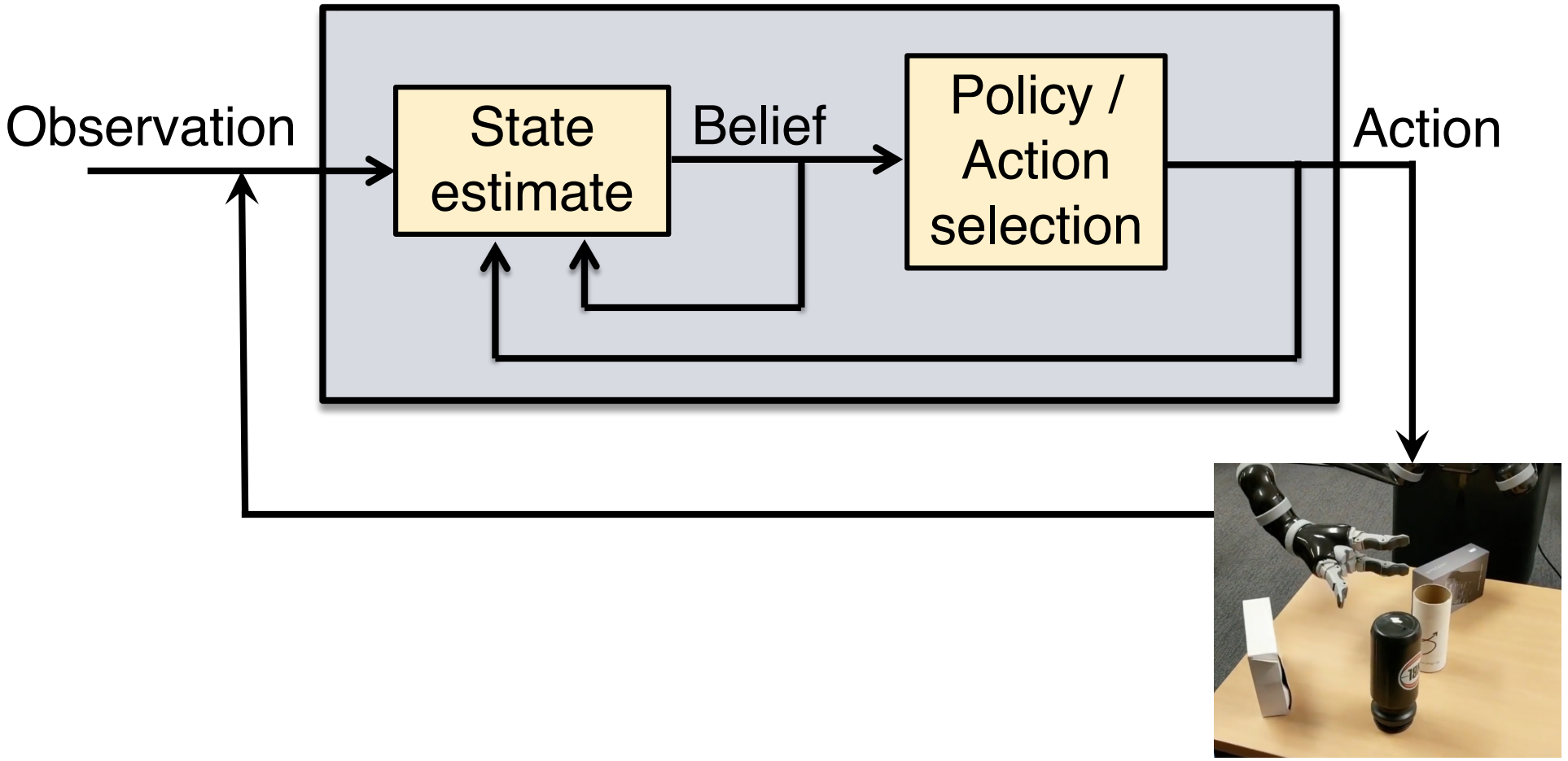
Partially Observable Markov Decision Processes (POMDPs)

Z(s', a, o): Observation function P(o | s', a) ; o ∈ O
s': States, a: action, o: Observation

T(s, a, s'): Transition function P(s' | s, a)
s, s': States, a: action

R: Reward function

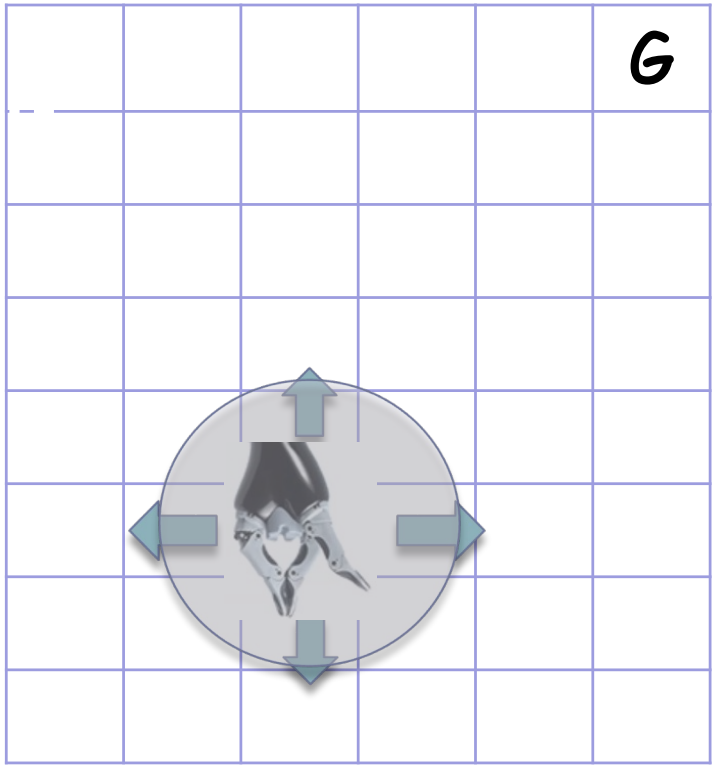
Partially Observable Markov Decision Processes (POMDPs)



Belief: Distribution over states
Solving a POMDP: Computing the best policy –maps beliefs to the best action

POMDP Model

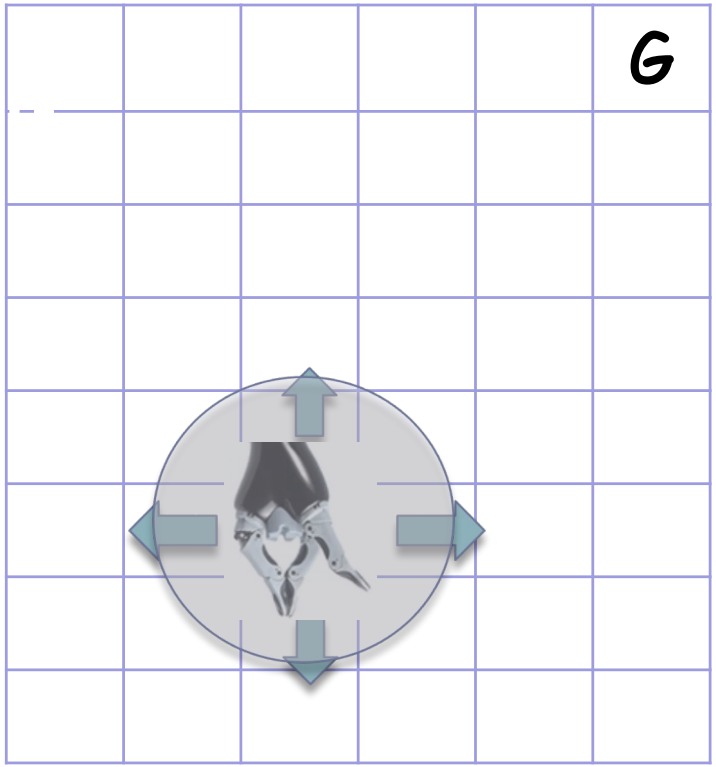
- Main components:
  - State space (S)
  - Action space (A)
  - Observation space (O)



POMDP Model

- Main components:
  - State space (S)
  - Action space (A)
  - Observation space (O)
  - Transition function (T)
    - T = P(s' | s, a)
  - Observation function (Z)
    - Z = P(o | s', a)
  - Reward function (R)
    - R(s, a)

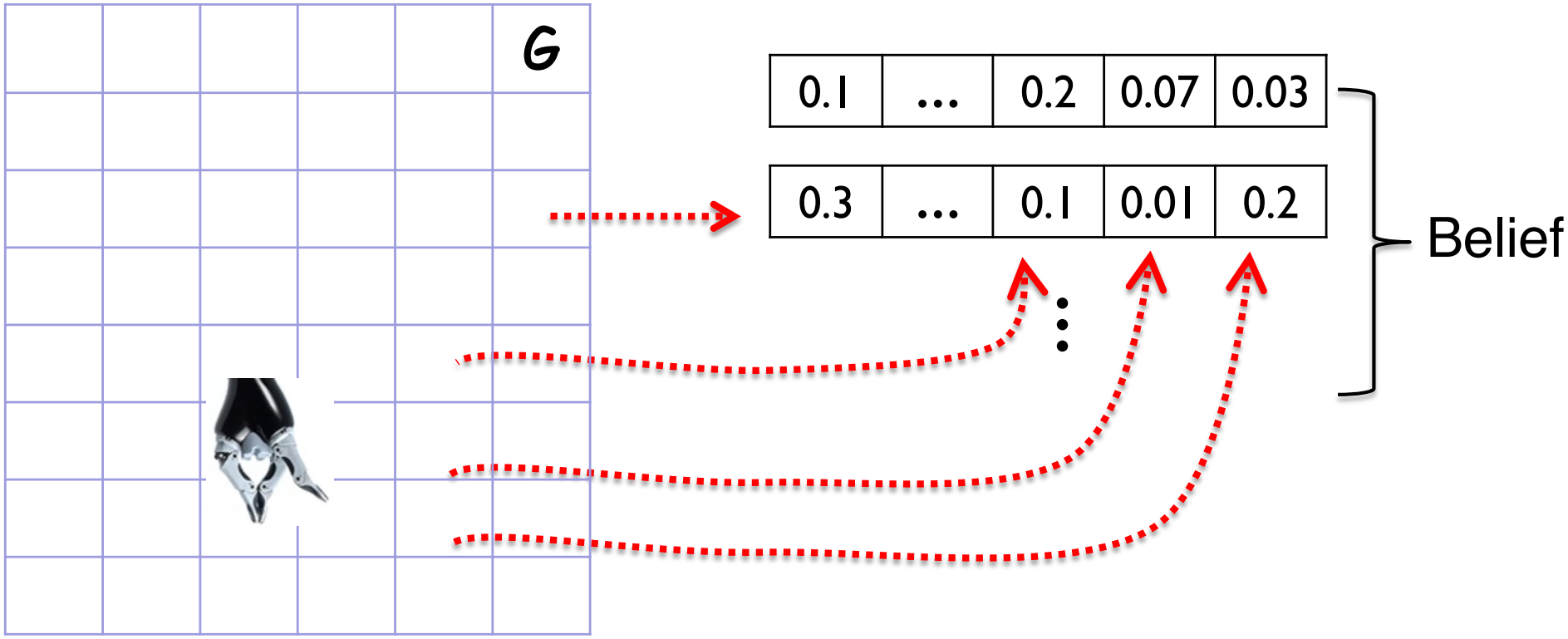
Not known



The states, actions, & observations can be continuous spaces

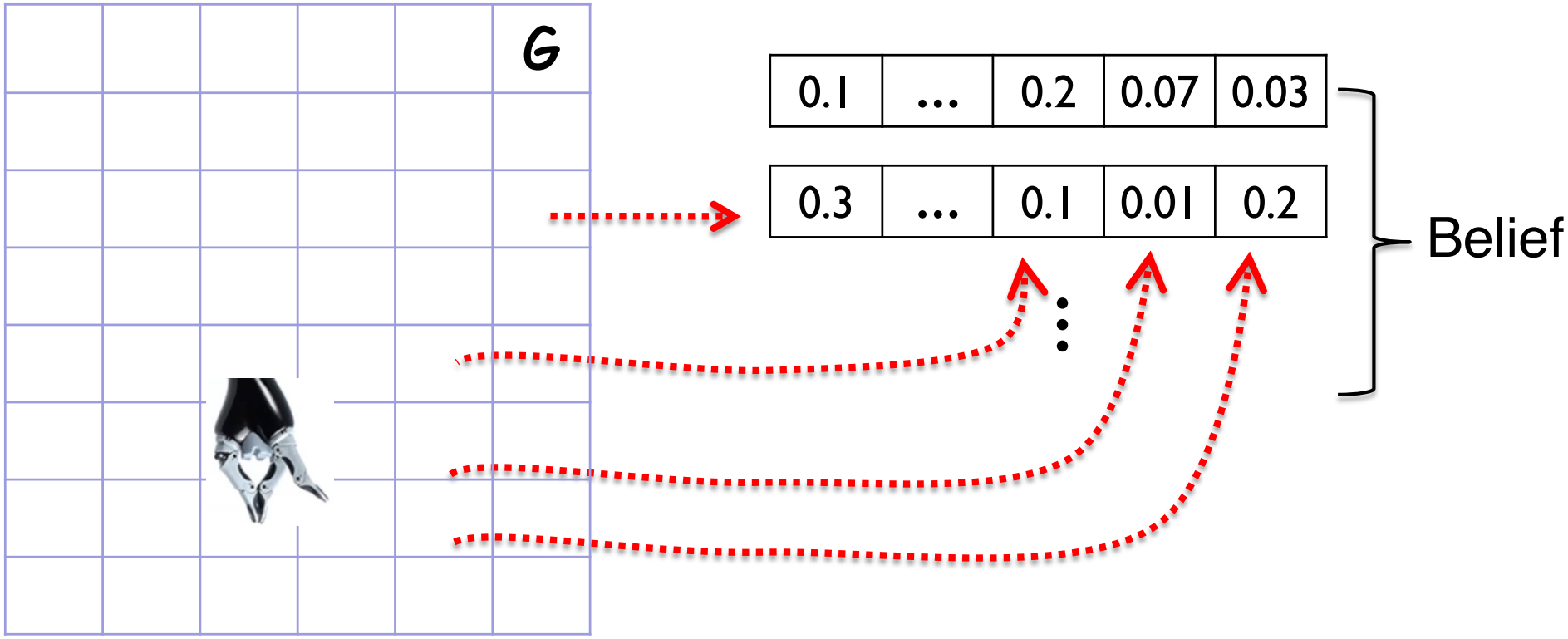
$s, s' \in S, a \in A, o \in O$

Key to POMDPs: Belief



- A belief: a distribution over the state space
- Belief space: The set of all beliefs.
  - What's the dimension of the belief space?

Key to POMDPs: Belief



- Belief: distribution over the state space
  - The belief space is a simplex with dim = |S|-1
- Policy: mapping from beliefs to actions

Key to POMDPs: Belief

- A belief is a sufficient statistics of the entire history of actions performed and observations perceived
- Hence, the 1st order Markov in POMDP is only in its model (transition and observation function)
- A POMDP policy is computed wrt to the entire history because it is computed wrt beliefs

Best policy

- Maps each belief to an action that maximizes the following objective function

$$V^*(b) = \max_{a \in A} \left(\sum_{s \in S} R(s,a) b(s) + \gamma \sum_{o \in O} P(o|b,a) V^*(b') \right)$$

Expected immediate reward

Expected total future reward

b': next belief after the system at belief b performs action a and observes o

γ: discount factor $(0,1)$

Computationally intractable [Papadimitriou & Tsikilis'87, Madani, et.al.'99].

Relation between MDPs & POMDPs

- A POMDP can be viewed as an MDP in the belief space

$$V^*(b) = \max_{a \in A} \left(\sum_{s \in S} R(s,a) b(s) + \gamma \sum_{o \in O} P(o|b,a) V^*(b') \right)$$

Immediate reward in the belief space: R(b,a)

Total future reward in the belief space: $\sum_{o \in O}$ P(b' l b, a, o)V(b')

- MDP can be viewed as as a degenerate POMDP

Formulation for b' and $P(o|b,a)$?

- Mathematically,
  - Use Bayes rule

$$
\begin{aligned}
b'(s') &= P(s'|o,a,b) \\
&= \frac{P(o|s',a,b)P(s'|a,b)\cancel{P(a,b)}}{P(o|a,b)\cancel{P(a,b)}} \\
&= \frac{P(o|s',a)\sum_s P(s'|s,a)b(s)}{\sum_{s''} P(o|s'',a)\sum_s P(s''|s,a)b(s)}
\end{aligned}
$$

The denominator $P(o|a,b)$ is essentially the normalizing factor that makes b' sums to 1.

This session

- Frameworks for planning under uncertainty
  - ✓Markov Decision Processes (MDPs)
  - ✓Partially Observable MDPs (POMDPs)
  - Reinforcement Learning
- Offline Solving: Value Iteration
  - Value Iteration for MDP
- Online Solving: MCTS-based
  - MDP

Intuitively,

- Reinforcement learning (RL) is a machine learning approach that learns by doing
  - The agent is not provided with a model of how the robot's world works and/or which states are good
  - Rather, the agent learns by trial & error, and evaluation of states, actions, and their outcomes

Formally,

- An RL is an MDP where the transition and/or reward functions are not initially known
  - S: State space.
  - A: Action space.
  - T: transition function.

    $T(s, a, s') = P(S_{t+1} = s' \mid S_t = s, A_t = a)$.
  - R: Reward function.

    $R(s, a)$.
- Need to try & explore

At least one of these is not known

Solving?

- Two broad approaches
  - Model free: Learn the policy / value function directly
  - Model-based: Learn the MDP model, and then solve the MDP

RL can be framed as a POMDP
aka. Bayesian Reinforcement Learning

RL: MDP with missing components

- State space (S_{MDP})
- Action space (A_{MDP})
- Transition function (T_{MDP})
- Reward function (R_{MDP})

Not known

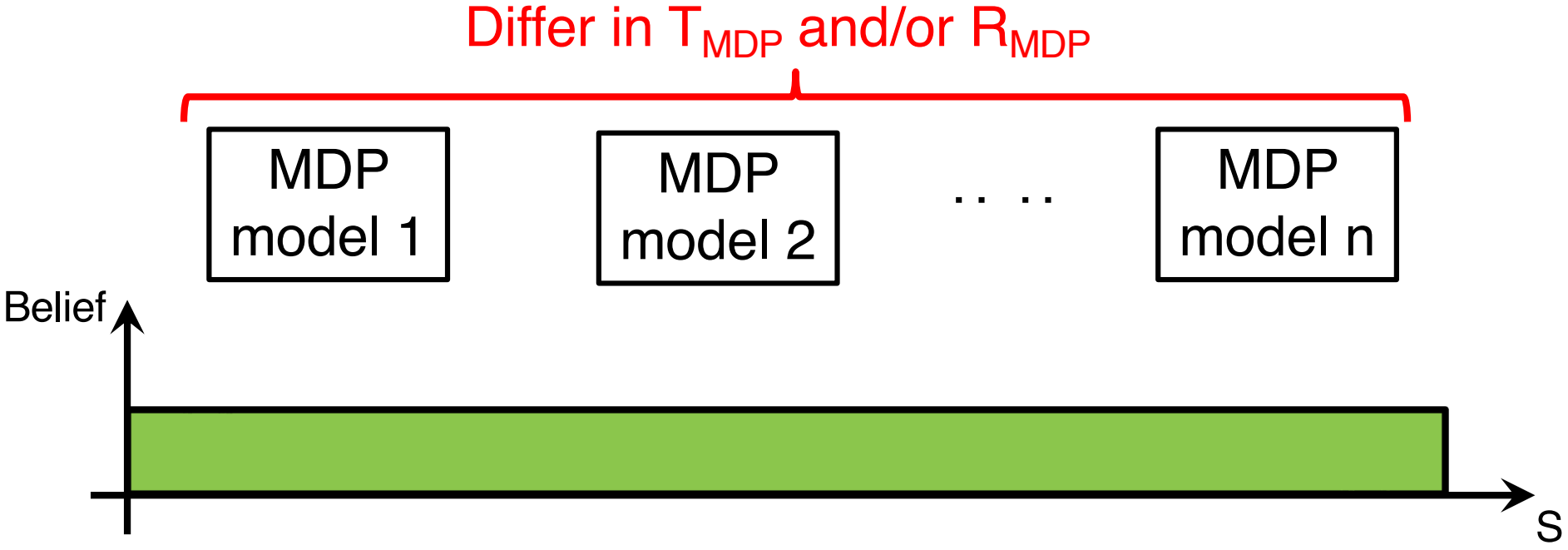
Construct a POMDP where

- The states are MDP states X parameters of the T_{MDP} & R_{MDP}
- Essentially, partial observability on which MDP model is the right model
- A, T, R follows from the particular MDP model
- O & Z are observations & observation function about which MDP model is correct

Bayesian RL

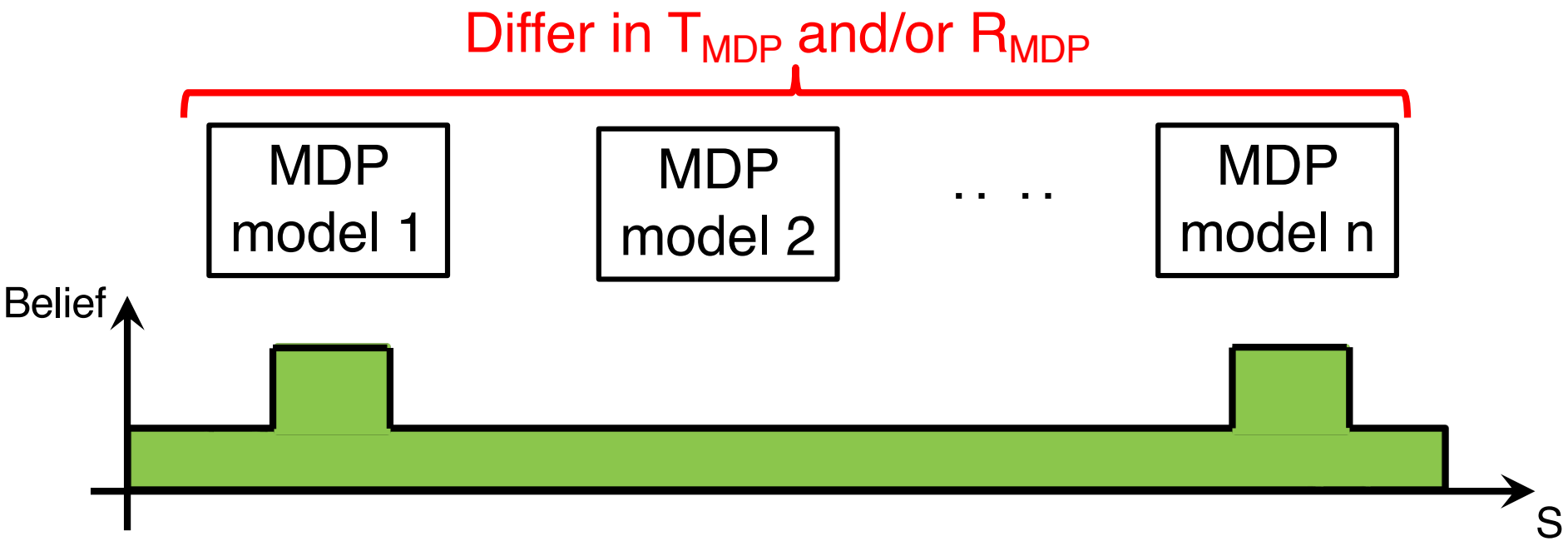
RL can be framed as a POMDP aka. Bayesian Reinforcement Learning

- Beliefs in POMDP becomes distribution over the possible MDP models



RL can be framed as a POMDP aka. Bayesian Reinforcement Learning

- Belief update means updates in the agent's understanding on which model is correct



POMDP automatically balances the trade-off between generating accurate model and to achieve the goal

In many cases, can achieve the goal without knowing the exact model

In a nutshell…

- MDP:
  - Non-deterministic action effects + fully observable
  - A degenerate POMDP
- POMDP:
  - Non-deterministic action effects + partially observable
  - MDP in the belief space
- RL:
  - MDP without transition and/or reward functions
  - Problem-wise, can be viewed as a POMDP, where partial observability is caused by incomplete information about the underlying MDP problem.

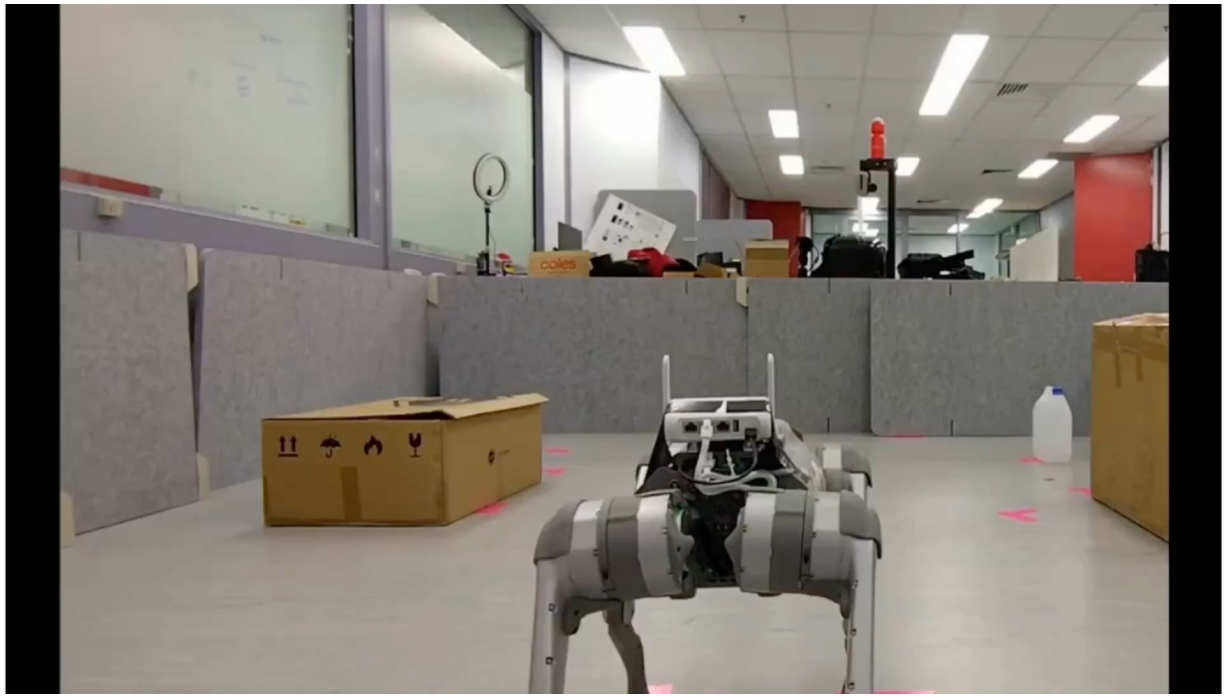
Why bother?

- Why not just give the data and learn the policy?

- The structure helps learning

A Recent Work:
Model Learning + POMDP Solving on GPU

Target Finding in
Uncertain Environment



3x speed-up

Learn from simulation only, run direct on physical robot 10/10

Compare to Recurrent SAC (SOTA model free), use < 10% data for training, equal performance on seen environment, up to 7x higher success rate on unsee environments

M. Hoerger, M.R. Sudrajat, and H. Kurniawati. Vectorized Online POMDP Planning. ICRA'26
M. Hoerger, R. Joshi, R. Shome, I. Manchester, and H. Kurniawati. VOiLA: Vectorized Online planning wIth Learned diffusion model for POMDP Agents. Under review.

This session

- ✓Frameworks for planning under uncertainty
  - ✓Markov Decision Processes (MDPs)
  - ✓Partially Observable MDPs (POMDPs)
  - ✓Reinforcement Learning
- Offline Solving: Value Iteration
  - Value Iteration for MDP
- Online Solving: MCTS-based
  - MDP

Offline Solving

- Compute the entire policy before execution
- During execution, only need use the policy
  - Can think of the policy as a lookup table
  - Though, often using a more efficient representation than lookup table

Value Iteration for MDP

- Iterate calculating the optimal value of a state until convergence.
- Algorithm:

  Initialize $V^0(s) = R(s,a)$ for all s

  Loop

  For all s {

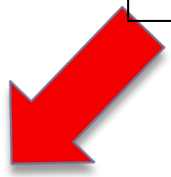
  $$V^{t+1}(s) = \max_a \left(R(s,a) + \gamma \sum_{s'} T(s,a,s') V^t(s') \right)$$

  }

  t = t + 1

  Until $V^{t+1}(s)=V^t(s)$ for all s (impl: $\max_s |V^{t+1}(s)-V^t(s)| < 1e\text{-}7$)

Often called value update or Bellman update or Bellman backup.



- Will it converge to V*? Can we have a bound on how long / how many iterations to converge?

Value Iteration – Approximately Optimal

- No guarantee we'll reach optimal in finite time.
- However, it's guaranteed we can get close to optimal within finite time. In fact, polynomial in |S|, |A|, 1/(1-γ).
- If we want to ensure that we are ε close to optimal, then #iterations should be

$$\left\lceil \frac{\log\left({}^{2R_{max}}/_{\varepsilon(1-\gamma)}\right)}{\log\left({}^{1}/_{\gamma}\right)} \right\rceil$$

Value Iteration – Issues

- Issues when the state space is large?
  - Computing Bellman update at every steps
  - Policy representation

This session

- ✓ Frameworks for planning (sequential decision-making) under uncertainty
  - ✓ Markov Decision Processes (MDPs)
  - ✓ Partially Observable MDPs (POMDPs)
  - ✓ Reinforcement Learning
- ✓ Offline Solving: Value Iteration
  - ✓ Value Iteration for MDP
- Online Solving: MCTS-based
  - MDP

Online Solving

- Online Solving: At every step, the agent is given a short amount of time to compute the best action for the current step
  - Less memory requirements
  - A bit slower during execution, though these days, we can compute even good POMDP policy with 1 sec planning time per step for 6-DOFs manipulator
- Does not need explicit model
  - Use a generative model (sampler, can be a simulator)
  - Computes a good policy by interacting with the simulator
- Assume: Initial state/belief is known

Monte Carlo Tree Search (MCTS)

- Combines tree search & Monte Carlo
- Monte Carlo
  - A sampling-based method to approximate the value of complicated functions, i.e., the ones that are too difficult/time consuming to compute exactly.
  - In MDP, can be used to approximate the expected total future reward if the agent follows an optimal policy.

$$V_{\pi^*}(s) = R(s,a) + \gamma \sum_{s'} T(s,\pi(s),s') V_{\pi^*}(s')$$

Kocsis, Levente, and Csaba Szepesvári. "Bandit based Monte-Carlo Planning." ECML'06

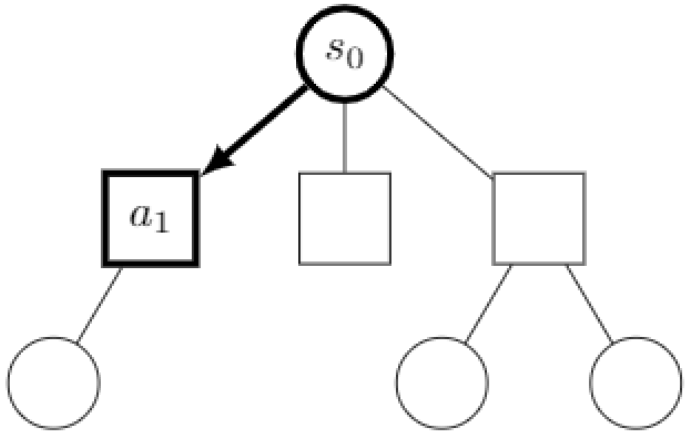
Monte Carlo Tree Search (MCTS)

- AND-OR Tree
  - OR-nodes represent states & associated value estimates
  - AND-nodes represent actions & associated Q-value estimates

- Wait, why do we need AND-OR Tree for MDP, which is fully observed?

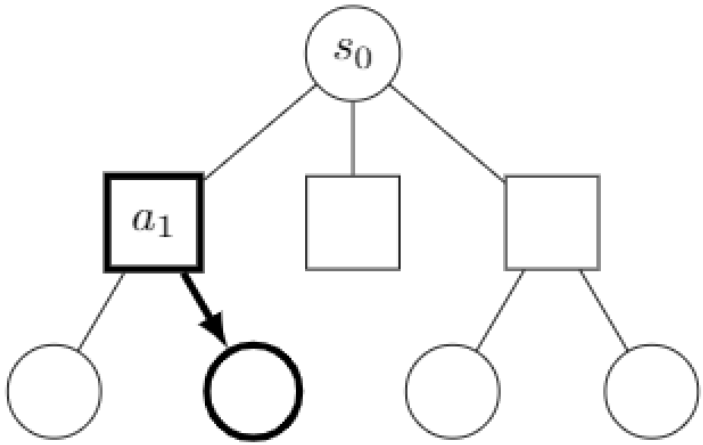
Monte Carlo Tree Search (MCTS)

- Build the search tree based on the outcomes of forward simulations
- Iterate over 4 main components:
  - Selection: Choose the best path



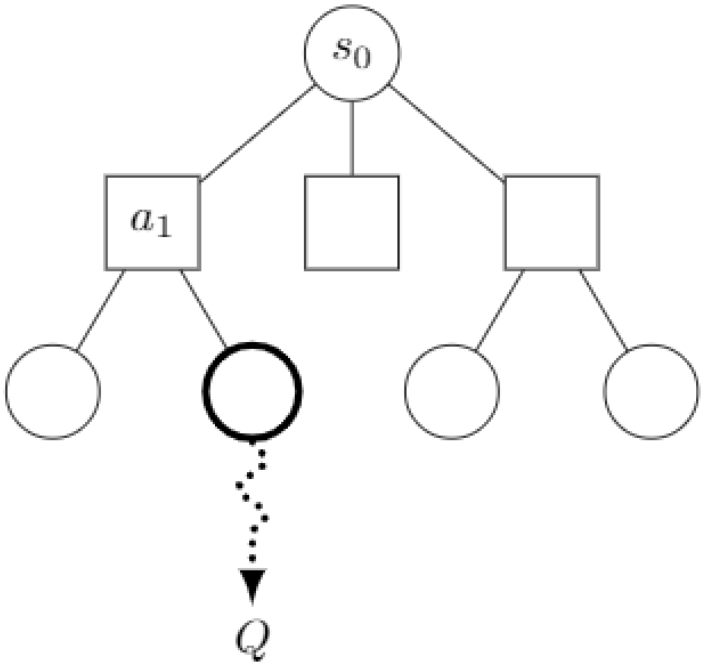
Monte Carlo Tree Search (MCTS)

- Build the search tree based on the outcomes of forward simulations
- Iterate over 4 main components:
  - Selection: Choose the best path
  - Expansion: When a terminal node is reached, add a child node



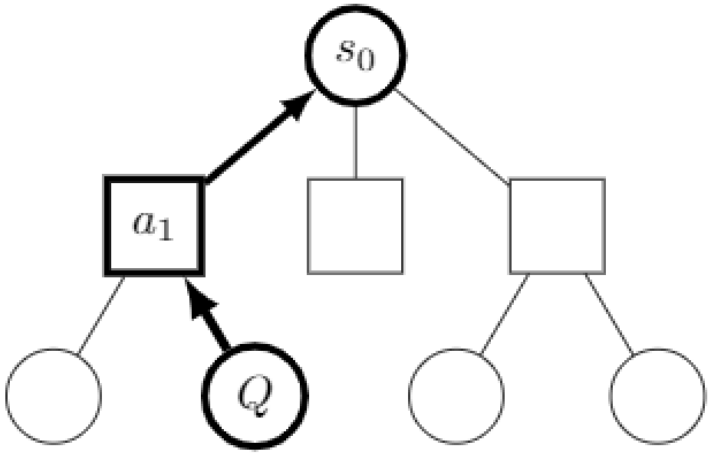
Monte Carlo Tree Search (MCTS)

- Build the search tree based on the outcomes of forward simulations
- Iterate over 4 main components:
  - Selection: Choose the best path
  - Expansion: When a terminal node is reached, add a child node
  - Rollout: Estimate the value of the newly added node, usually via simulation but can also use learned functions



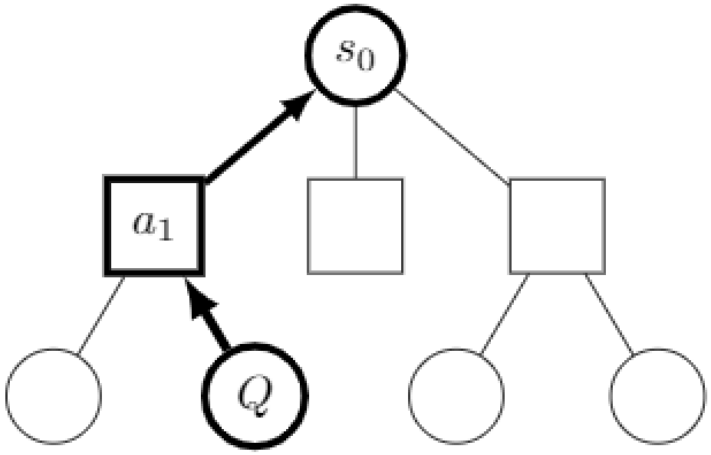
Monte Carlo Tree Search (MCTS)

- AND-OR Tree
- Build the search tree based on the outcomes of forward simulations
- Iterate over 4 main components:
  - Selection: Choose the best path
  - Expansion: When a terminal node is reached, add a child node
  - Rollout: Estimate the value of the newly added node, usually via simulation but can also use learned functions
  - Backpropagation: Update the value of the nodes visited in this iteration



Commonly Used MCTS for MDP

- AND-OR Tree
- Build the search tree based on the outcomes of forward simulations
- Iterate over 4 main components:
  - Selection: Choose the best path
  - Expansion: When a terminal node is reached, add a child node
  - Rollout: Estimate the value of the newly added node, usually via simulation but can also use learned functions
  - Backpropagation: Update the value of the nodes visited in this iteration



Node Selection

- Multi-armed bandit to select which action to use
  - In general, use a method called Upper Confidence Bound (UCB1).
  - Choose an action a to perform at s as:

$$\pi_{UCT}(s) = \arg\max_{a \in A} Q(s,a) + c\sqrt{\frac{\ln(n(s))}{n(s,a)}}$$

Exploitation Exploration

c: A constant indicating how to balance exploration & exploitation, need to be decided by trial & error.

n(s): #times node s has been visited.

n(s,a): #times the out-edge of s with label a has been visited.

- MCTS + UCB is often called Upper Confidence Bound for Trees (UCT)

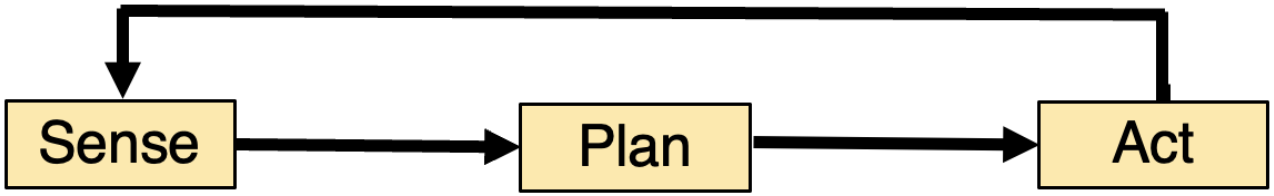
This session

- ✓ Frameworks for planning (sequential decision-making) under uncertainty
  - ✓ Markov Decision Processes (MDPs)
  - ✓ Partially Observable MDPs (POMDPs)
  - ✓ Reinforcement Learning
- ✓ Offline Solving: Value Iteration
  - ✓ Value Iteration for MDP
- ✓ Online Solving: MCTS-based
  - ✓ MDP

What's Next?

- Today hands-on:
  - Motion Planning
- Tomorrow workshop:
  - Motion planning with kinodynamic constraints
  - RL for low-level control
  - Integrating motion planning & low-level control

Take Home Message



When planning planning accounts for uncertainty, we can relax requirements for accuracy in model, which in turn reduce data & time to learn the model

Many types of uncertainty, but for planning, accounting for uncertainty in the effects of actions and in the observations are sufficient to cover the various types uncertainty

Of course, if we have more accurate model, we should be able to do more…